

Laboratory 7

Variant 2, Group 32

by Adam Shahumi and Santiago Puig-Terrero López

1. Introduction

The objective of the task is to calculate and return the exact number of days (N) between two provided dates. The implementation is built upon two strict assumptions provided in the instructions: the calendar year is 2024 (which is a leap year), and the maximum difference between the given dates will be $N \leq 366$ days. The program expects inputs as string literals formatted as "DDMM" (for example, "2704" representing the 27nd of April), and the expected output is a single integer representing the calculated interval.

2. Logic flow

To determine the difference between two dates, the solution avoids complex chronological step-by-step counting. Instead, it relies on an absolute reference point logic. It maps each string date to its absolute day of the year (an integer ranging from 1 to 366). By calculating this absolute calendar day for both the first input date and the second input date, the complex calendar problem is simplified to finding the absolute mathematical difference between two integers. This approach ensures highly efficient calculation and inherently supports calculating the correct interval regardless of chronological order (as in, it will output the correct distance whether Date1 occurs before or after Date2).

3. Code summar

The Prolog solution is structured around three main components to execute the logic flow:

- **cum_days - Knowledge Base:** This is a set of Prolog facts that act as a lookup table. Because the instructions mandate assuming the year is 2024, these facts explicitly account for a leap year. Each fact maps a specific month to the total number of days that have elapsed in the year prior to the 1st of that month (e.g., `cum_days(3, 60)` signifies that 60 days have passed before March 1st, accounting for 31 days in January and 29 days in February).
- **date_to_days - Helper Rule:** This rule handles all parsing and data type conversion. It utilizes the built-in `sub_string` predicate to split the unified "DDMM" string into separate "Day" and "Month" strings. Next, it uses `number_string` to cast these substrings into usable integer variables. Finally, it queries the elapsed days

from the `cum_days` facts and uses the `is` operator to sum the elapsed days with the day of the current month, returning the absolute day of the year.

- **Interval - Main Rule:** This serves as the primary query interface requested in the exercise instructions. It invokes the helper rule for both provided date strings, calculating `Days1` and `Days2`. It then calculates the final interval utilizing the absolute value function (`Diff is abs(Days2 - Days1)`) and prints the result to the console.

4. Challenges and Testing

Rather than facing significant development hurdles, the primary focus was placed on identifying and checking specific test cases to guarantee the code's absolute accuracy and robustness against bugs.

The solution was systematically verified using the following test categories:

- **Standard Lab Examples:** The code was verified against the exact example queries provided in the prompt, successfully confirming that `interval("2205", "0506")` outputs 14 and `interval("0102", "1102")` outputs 10.
- **Reverse Chronology:** Queries were tested where the second date occurs before the first date (e.g., entering the June date first and the May date second). The integration of the `abs` function successfully ensured that the interval remains a positive integer regardless of input order.
- **Leap Year Boundary Verification:** Test cases spanning across late February and early March were checked to ensure the program accurately factored in February 29th, preventing any off-by-one errors.
- **Identical Dates:** Passing identical strings into the predicate was tested to confirm that the system correctly evaluates the interval to 0.